

Software Security Testing via Misuse Case Modeling

Samer Khamaiseh

Department of Computer Science
Boise State University
Boise, ID 83725, USA
samerkhamaiseh@u.boisestate.edu

Dianxiang Xu

Department of Computer Science
Boise State University
Boise, ID 83725, USA
dianxiangxu@boisestate.edu

Abstract—Software security testing is an important technique for discovering software vulnerabilities that violate security requirements. Existing security testing methods, however, seldom generate security tests directly from security requirements specifications. To address this issue, this paper presents an approach for constructing security test models from the artifacts of misuse case modeling (i.e., use/misuse cases and mitigation use cases), which is a popular method for security requirements specification in the software development process. The security test models can then be used to automatically generate security tests, which consist of test inputs (normal behaviors from use cases and attack actions from misuse cases) and test oracles from mitigation use cases. We have applied the approach to two case studies. One case study demonstrates that the proposed approach can build security test models in a structured fashion such that the generated security tests are as effective as reported in the literature. The second case study applies the proposed approach to an ongoing software development project. The security tests have revealed at least 24 vulnerabilities, and are very helpful for the development team to improve the security of the software implementation. This demonstrates that the proposed approach is effective in the software development process.

Keywords—security testing; software security, use case, misuse case; mitigation use case; modeling

I. INTRODUCTION

Software vulnerability is a major cause of the widespread cyber security problems [1]. Vulnerabilities get into software for various reasons such as lack of security requirements specification, flawed software design, insecure programming, and inadequate quality assurance. It is well-recognized that software security should not be treated as an afterthought; instead, it should be handled carefully throughout the software development process [2]. A common practice for revealing vulnerability during the software development process is testing for security, which aims to find potential vulnerabilities by exercising the software under test with tests.

Testing for security is time-consuming because a software system often deals with too many possible inputs. It is also challenging because it needs to focus on the potential “presence of an intelligent adversary bent on breaking the system” [3]. Thus, security testing with threat models has recently emerged as a viable option for automated generation and/or execution of security tests. Wang et al., [4] have proposed to create security tests from threat models specified by UML sequence diagrams. Marback et al. [5] have

developed a method for generating security tests as attack paths from attack trees. These approaches are useful after the software under test has been implemented because the threat models involve specific implementation details of the software. Xu et al. have developed the MISTA tool for automatically generating security tests from formal threat models [6][7]. Threat models specified as Predicate/Transition (PrT) nets are built in a way that is independent of implementation details. The executable test code of security tests can be generated once the elements of threat models are mapped into implementation constructs through a MIM (Model-Implementation Mapping) specification. In MISTA, the separation of threat models from implementation details makes it feasible to apply threat modeling and testing at different software development phases. Although studies have shown that MISTA is an effective tool for security testing [6][7], it can be a challenge to construct formal threat models. MISTA itself does not prescribe specific methodologies for creating threat models from the artifacts of software development activities, particularly in the early phases of secure software lifecycle such as security requirements analysis.

This paper presents an approach for building threat models as PrT nets (called security test models) from the artifacts of security requirements analysis so that executable code of security tests can be generated automatically by the MISTA tool. As shown in Figure 1, the approach creates security test models from the specifications of use/misuse cases, and mitigation use cases produced by misuse case modeling. The security test models are then used as inputs to MISTA for generation of security tests. Misuse case modeling [8][9] is an effective technique for the specification of security requirements in the early stage of software development process. It is an extension to use case modeling, which is a popular approach to functional requirements specification [10][11]. Misuse case modeling elicits functional and security requirements in terms of use/misuse cases and mitigation use cases. They provide essential requirements specification for subsequent development activities such as system design, implementation, and validation. When the system is implemented, it is important to validate that the system implementation does meet the specified security requirements. However, it remains open how we can use misuse cases and mitigation use cases for such validation. The proposed approach aims to address this issue.

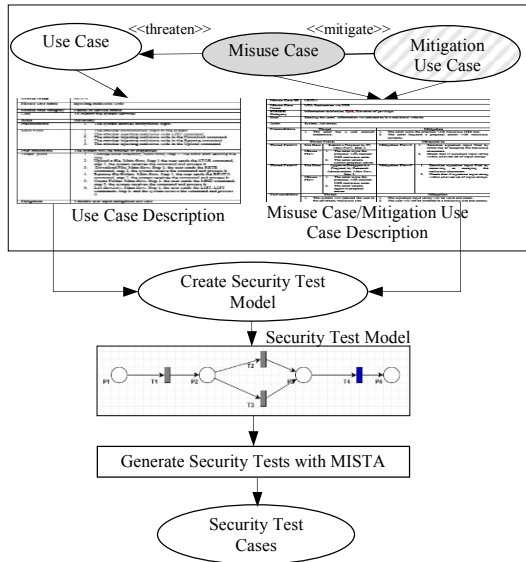


Figure 1. From misuse case modeling to security testing

To evaluate the proposed approach, we have applied it to the FileZilla FTP Server case study in [6]. This case study aims at demonstrating that the proposed approach can produce security test models from use/misuse cases and mitigation use cases and that the security tests resulted from these models are as effective as reported in [6]. The difference is that our approach builds security test models from requirements specification in a structured fashion. To show that our approach is useful in the software development process, we have also applied it to an ongoing software development project, GPMS (Grant Proposal Management System), implemented by independent developers. The security tests have revealed at least 24 vulnerabilities in the implementation. The tests are being used by the development team to improve the implementation.

The rest of this paper is organized as follows. Section II provides an overview of misuse case modeling. Section III describes our approach for building security test models from use cases, misuse cases, and mitigation use cases. Sections IV and V present the case studies on FileZilla FTP Server and GPMS, respectively. Section VI summarizes related work. Section VII concludes this paper.

II. MISUSE CASE MODELING

To make the paper self-contained, this section briefly overviews the misuse case modeling approach. Generally,

misuse case modeling consists of three iterative steps: (1) use case modeling, (2) specification of misuse cases, and (3) specification of mitigation use cases.

Use case modeling aims to create use cases that represent the required functions of the software under development. They are specified by use case diagrams and textual descriptions. A use case diagram describes actors, use cases, relationships between system actors and use cases, and relationships among use cases. In this paper, we follow the commonly used template of use case descriptions [10][11]. It consists of use case name, actor, goal, precondition, main flow, post-condition, extension point, alternative flow, and exception flow. A precondition specifies the constraint before the use case can be applied. Postcondition describes the outcomes after a successful conclusion of the use-case. Main flow describes the success scenario is the main story-line of the use-case. Alternative flow specifies alternative scenarios, including endpoints and shortcuts. Exception flow deals with exceptions, errors, and failures. Extension points refer to the points where the use case can be extended by other use cases. Table 1 shows the description of use case “submit a proposal by principal investigation” in the case study GPMS (Grant Proposal Management System), which will be detailed in section VI.

TABLE I. DESCRIPTION OF USE CASE “SUBMIT A PROPOSAL”

Use case #	UC-1
Use case name	Submit a proposal by principal investigator (PI).
Actor	Principal Investigator (PI).
Goal	Submit a proposal to department chair for approval
Precondition	1. The PI created the proposal and saved it. 2. The Co-PI(s) signed the proposal. 3. The proposal status not submitted.
Main Flow	1. The actor login to the account. 2. The actor selects “My proposals” action. 3. The actor selects the proposal by selecting the edit proposal action. 4. The system opens the proposal in edit mode. 5. The actor signs the proposal by inserting a signature, date, and comment. 6. The actor selects the submit action. 7. The system sends a notification to the department chair, PI, Co-PI(s) and Senior Personnel. 8. The system records the request in the user audit log.
Post-Condition	1. The proposal status changed to waiting for chair approval. 2. The actor has read access to the proposal.
Extension Points	1. Step 7, extends Notify Users use case.
Alternative flow	2.a The actor uses the search engine 2.a.1 The actor inserts the proposal information in the search fields. 2.a.2 The system returns the search result. 2.a.3 The actor selects the proposal. The use case continues at the actor selects the submit action in Main Flow
Exception flow	4.a CO-PI has not signed the proposal 4.a.1 The system shows an error message that CO-PIs have not signed on the proposal.

Based on the use cases, we examine the components (main flow, alternative flow, extension points, and exception flow, etc.) of each use case so as to identify whether they can be

misused to violate security goals. We also use the STRIDE threat category to suggest potential misuses [12]. STRIDE stands for Spoofing identity, Tampering data, Repudiation, Information disclosure, Denial of service, and Elevation of privilege. It covers a variety of known attacks that violate common security requirements.

Note that multiple misuse cases, and thus mitigation use cases, may apply to one or more use case. Figure 2 shows a sample diagram that consists of one use case “Submit a Proposal by PI”, two misuse cases “URL Redirection” and “Disable Save Action”, and two mitigation use cases “Prevent URL Redirection” and “Prevent Disable Save Action Attack”. Both misuse cases threaten the use case “Submit a proposal by PI”. The misuse case “URL Redirection” is mitigated by the “Prevent URL Redirection”, whereas the misuse case “Disable Save Action” is mitigated by “Prevent Disable Save Action Attack”.

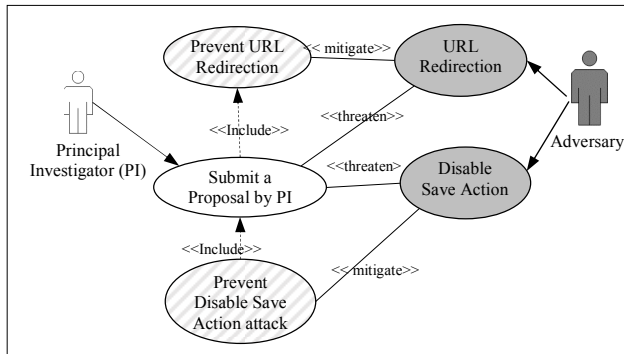


Figure 2. A use/misuse case, mitigation use case diagram

TABLE II. DESCRIPTION OF MISUSE “URL REDIRECTION” AND ITS MITIGATION

ID	MUC-1			
Name	URL Redirection			
Category	Information disclosure, DoS, Elevation of privilege.			
Goal	Steal user credentials via redirection to a malicious website.			
Actor	System, Adversary			
Pre-conditions	Threat		Mitigation	
	The actor has credentials.		The actor signs the proposal with malicious XSS code.	
Threats			Mitigated by	
Threat 1	Point	Use Case: Submit a Proposal by PI, Main Flow, Step 5.	Mitigation Flow-1	1. Sensitize signature input by escaping undesirable characters. 2. Check if signature input string within allowed set of inputs
	Attack Flow	1. The actor signs the proposal with redirect XSS malicious code. 2. The actor selects submit proposal action		
Threat 2	Point	Use case: create a Proposal by PI, Main Flow, Step 13.	Mitigation Flow-2	Same as Mitigation Flow-1
	Attack Flow	1. The actor signs the proposal with redirect XSS malicious code. 2. The actor selects save proposal action		

Post-conditions	1. The system will redirect the user to the adversary malicious site.	1. The signature input string will be valid and clean. 2. The user will be directed to the correct site.
-----------------	---	---

Misuse cases together with corresponding mitigation use cases are described in a tabular form, including name, STRIDE category [12], goal of misuse (attack), actor, preconditions of misuse and mitigation, each threat point (the corresponding component of a threatened use case), attack flow, corresponding mitigation flow, and postconditions of misuse and mitigation. A threat point refers to a step in the main/alternative/exception flow of a use case, and an attack flow provides a step-by-step description of how the adversary may apply the threat. A mitigation flow describes the security requirements for countermeasure of the corresponding security threat. Table II shows the description of misuse case “URL Redirection” in Figure 2, together with the mitigation use case. Table III shows the description of misuse case “Disable Save Action” in Figure 2, together with the mitigation use case. We will use Figure 2 and Tables II and III as a running example of building security test models.

TABLE III. DESCRIPTION OF MISUSE “DISABLE SAVE ACTION” AND ITS MITIGATION

ID	MUC-2			
Name	Disable Proposal Save Action			
Category	DoS, Elevation of privilege.			
Goal	To prevent users from saving the proposal document.			
Actor	System, Adversary			
Pre-conditions	Threat		Mitigation	
	The actor has credentials.		The actor signs the proposal with malicious XSS code.	
Threats			Mitigated by	
Threat 1	Point	Use Case: Submit a Proposal by PI, Main Flow, Step 5.	Mitigation Flow-1	1. Sensitize signature input by escaping undesirable characters. 2. Check if signature input string within allowed set of inputs
	Attack Flow	1. The actor signs the proposal with disable save action XSS malicious code. 2. The actor selects submit proposal action		
Threat 2	Point	Use case: approve/disapprove a Proposal by the department chair, Main Flow, Step 7.	Mitigation Flow-2	Same as Mitigation Flow-1
	Attack Flow	1. The actor signs the proposal with disable save action XSS malicious code. 2. The actor selects approve proposal action		
Post-conditions	1. The users will not be able to use save proposal action.		1. The signature input string will be valid and clean. 2. The user can use save proposal action.	

III. BUILDING SECURITY TEST MODELS

This section first introduces the formalism of security threats and then describes the creation of threat models from

the textual descriptions of use/misuse cases and mitigation use cases.

In MISTA, a threat model is specified by a PrT net $\langle P, T, F, I, \Sigma, L, \varphi, M_0 \rangle$ [6], where:

- P and T are a set of places (i.e., predicates) and a set of transitions, respectively.
- F and I are a set of normal arcs and a set of inhibitor arcs (representing negation), respectively.
- Σ is a set of constants, relations, and arithmetic operations.
- L is a labeling function on arcs $F \cup I$. Each arc label is a tuple of variables and/or constants in Σ .
- φ is a guard function on T . The guard condition of each transition is constructed from variables and the elements in Σ .
- $M_0 = \bigcup_{p \in P} M_0(p)$ is an initial marking, where $M_0(p)$ represents tokens in place p .

The dynamic behavior of a PrT net is interpreted by transition firings and firing sequences. We refer the reader to [6] for more details. A PrT net is a threat model if it has at least one attack transition (also called sink transitions in MISTA). The firing of an attack transition is a successful attack or indicates a sign of vulnerability.

Creating threat models from the descriptions of use/misuse cases and mitigation use cases can be centered around either use cases or misuse cases. The use-case centered approach consists of the following steps:

- 1) For each use case, find all relevant misuse and mitigation use cases. As a running example, we consider the relevant misuse and mitigation use cases for the use case “Submit a Proposal by PI” are only those in Figure 2.
- 2) For each threat of the use case, create a PrT net for the steps from the beginning of the use case to the threat point. A threat point can be a step in the main flow, alternative flow, or exception flow of a use case description. When creating or expanding a PrT net for the steps from a use case or misuse case, we focus on the steps performed by the user or adversary, not system actor. The reason is that the steps of a system actor are usually implemented by the system, whereas the steps of a user or adversary represent inputs to the system. In essence, our approach follows a black box testing strategy. Consider threat 1 in Table II as an example. The threat point is step 5 in the main flow of the use case “Submit a Proposal by PI” in Table 1. Steps 1-3 are converted to three transitions “UserLogin”, “SelectProposalViewAction”, “OpenProposalDocument”. Step 4 is not converted into a transition because it is a system action. Step 5 is replaced with attack steps.
- 3) Expand the PrT net with the steps in the attack flow of the threat. For threat 1 in Table II, the attack flow consists of two sequential steps: “The actor signs the proposal with redirect XSS malicious code,” and “The actor selects submit proposal action.” These two steps lead to two

transitions “SignProposalWithURLRedirectionAttack” and “SubmitProposalByPI”.

- 4) Expand the PrT net with the effects, rather than the specific steps, of the mitigation flow. The mitigation flow represents security requirements that should be implemented by the system. It is used as a test oracle to determine whether or not the system is secured from the attack. If the security requirements are not implemented correctly, the execution of security tests will fail, or the attempt to attack the system will be successful. For threat 1 in Table II, the mitigation is converted into transition “URLRedirectionMitigationEffect”.
- 5) Compose the PrT nets created for the same use case in steps 2) through 4) into one PrT net. This net is the security test model for the use case. It consists of all threats against the use case. The composition will help reduce duplication in security test models and facilitate verifying security test models for assuring that the security models are correctly specified. For the use case “Submit a Proposal by PI”, we have two threats – threat 1 in Table II and threat 1 in Table III. In the above, we have discussed how threat 1 in Table II is converted into a PrT net. This is similar for threat 1 in Table III. Composing these PrT nets results in the security test model in Figure 3 as they share four transitions according to the use/misuse case descriptions. Note that threat 2 in Table II and threat 2 in Table III are not formalized here because they apply to a different use case.

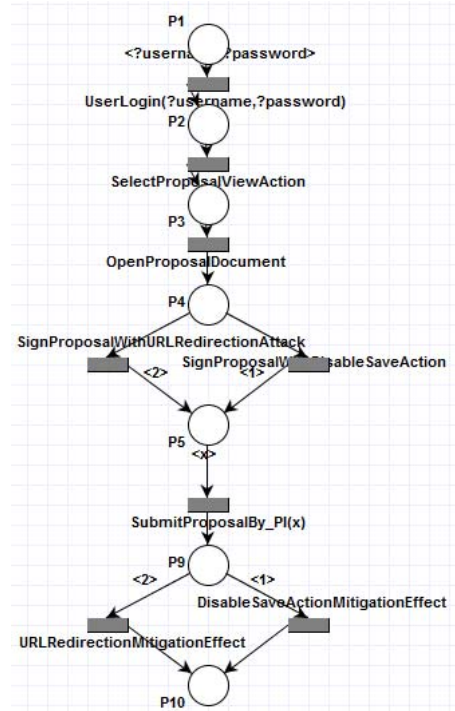


Figure 3. Sample security test model built from Figure 2 and Tables II and III.

- 6) Specify the Model Implementation Mapping (MIM) and Helper Code for the security test model for test code generation by MISTA. This paper will not elaborate on MIM and Helper Code. Details can be found in [7]. Steps 2) through 6) are repeated for all use cases.

PrT nets provide rich building blocks for specifying security test models, such as data processing and data flows, sequential events (actions), conditional (or choice) events, repetitive events, and concurrent events [7]. These blocks make it feasible to convert descriptions of use cases and misuse cases into PrT nets. The sequential, conditional, repetitive, and concurrent constructs in PrT nets are expressive enough to formalize the steps from the beginning of the use case to the threat point in the use case in 2) and the steps in the attack flow in 3). For 3), “Expand the PrT net with the steps in the attack flow of the threat” is implemented by adding the latter to the former by a sequential construct. In 5), “compose the PrT nets created for the same use case into one PrT net” is done by conditional constructs. For brevity, this paper will not elaborate on creation of places and arc labels.

In misuse case modeling, one misuse case may threaten multiple use cases. Thus, the misuse case centered approach works as follows: 1) for each misuse case description, find all relevant use cases; 2) for each threat in the misuse case, create a PrT net for the steps from the beginning of the use case to the threat point; 3) expand the PrT net with the steps in the attack flow of the threat; 4) expand the PrT net with the effects of the mitigation flow; 5) compose the PrT nets created for the same misuse case into one security test model. It consists of the same types of threats against all relevant misuse cases; 6) specify MIM and Helper Code for the security test model.

The use case centered approach and the misuse case centered approach can result in different representations of security test models. From the perspective of testing effectiveness, however, they should have the same capability of finding security vulnerabilities because they exercise the same set of security threats specified by the same set of misuse cases. This is confirmed by our case studies described in Sections IV and V. We have applied both approaches to our case studies. In theory, it is possible to integrate the use case centered approach and the misuse case centered approach by composing all use cases, misuse/mitigation use cases into one security test model. However, this is not practical because the test model would be too complex to manage.

It is worth pointing out that, to create complete security test models, we cannot simply rely on the attack flows in misuse case descriptions without information in the use case descriptions. Security exploits typically are malicious attacks or inadvertent misuse of system functions. In the use case and misuse case modeling approaches, these system functions are specified by use cases.

IV. CASE STUDY I: FILEZILLA FTP SERVER

This case study is a “replication” of the existing empirical study on security testing using formal threat models for the FileZilla FTP server, one of the most downloaded programs on Source Forge [6]. File Transfer Protocol (FTP) is a standard protocol used widely for transferring files between systems.

In [5], 35 out of the 38 vulnerabilities in the system were exposed by building security test models and generating security tests from these models. However, [5] did not provide guidance on how to build these security test models in a structured fashion. Our approach in this paper provides a framework for creating security test models, based upon use/misuse cases and mitigation use cases while exposing the same number of vulnerabilities.

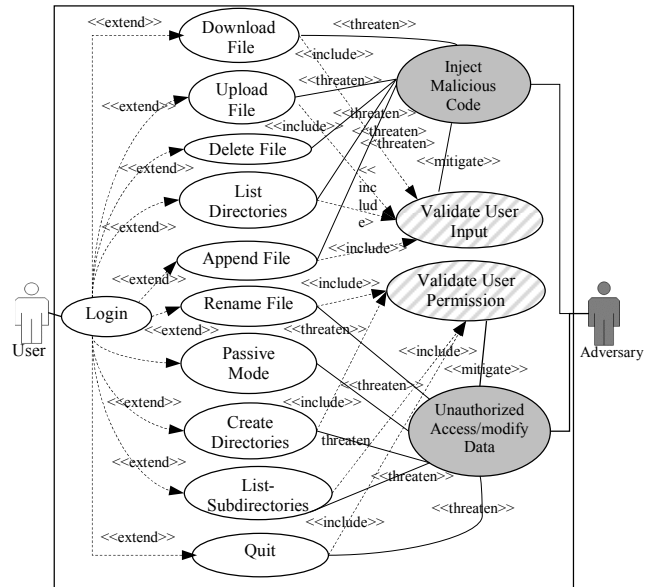


Figure 4. Use case/misuse case/mitigation use case diagram for FileZilla FTP Server

The FileZilla FTP Server used in [6] is version 0.9.34, which has 88,596 lines of C++ code in 107 classes. The eight threat nets are built according to the FTP specification published by the Internet Engineering Task Force (IETF) and STRIDE threats. The main FTP operations include login, connect, change directory, list directory, get current directory, change to passive mode, put file to server, get file from server, create directory, delete directory, delete a file, execute commands on server, SITE, rename a file, append a file, system, and quit. FileZilla FTP. The eight threat nets result in 76 security tests. To evaluate the effectiveness of these security tests, 38 mutants of the FileZilla Server 0.9.34 were created. Each mutant is a modified version of the original source code with one seeded vulnerability. The mutants cover the following types of vulnerabilities: buffer overflow, logic errors, password management errors, memory leak, format string, integer overflow, incorrect access control, and business logic flaws. The 76 security tests produced by the threat nets were able to kill 35 of the mutants. Memory leak, buffer overflow, and logic error are the vulnerabilities in the three remaining mutants. They are not handled by the threat nets.

In this paper, we used the most recent version of FileZilla FTP Server (i.e., 0.9.53) when the study was started. This version has 90,653 line of C++ code and 123 classes. Even though it is a newer version, we were able to create all the 38

mutants as in [6]. Our case study follows the approach described in Sections III and IV. That is, we created use/misuse cases and mitigation use cases according to the FTP specification and converted them into security test models. Figure 4 shows the use case/misuse case/mitigation use case diagram for FileZilla FTP Server. It consists of 11 use cases, two misuse cases, and two mitigation use cases. The two misuse cases are generic and apply to all misuse cases. 11 security test models are created from the use cases, misuse cases, and mitigation use cases. Figure 5 shows two possible attacks against the “Download File” use case: inject malicious code that leads to DoS and downloads a file without permission. Note that, here the security test models are organized with respect to use/misuse cases and mitigation use cases, whereas the formal threat model in [6] are organized with respect to STRIDE attacks against FTP functions. Nevertheless, the 22 security tests in this case study killed the same 35 mutants as in [6]. Compared to [6], our new case study implies a different way to build formal threat models through misuse case modeling. It has achieved the same effectiveness with a smaller test suite.

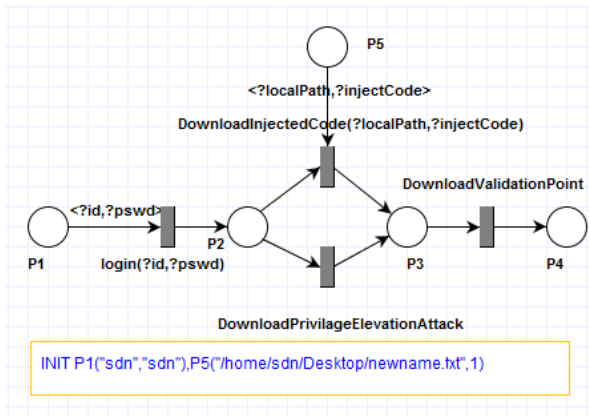


Figure 5. A security test model in the FileZilla FTP Server case study

V. CASE STUDY II: GPMS

Our second case study aims to demonstrate that our approach is useful for security testing in the software development process. The subject is an ongoing open source project, GPMS (Grant Proposal Management System), implemented by independent developers [13].

GPMS is a web-based application for managing the workflow of grant proposal submissions within an academic institution. Generally, a faculty member who plans to submit a grant proposal as the principal investigator (PI) initiates the workflow by creating and editing a new proposal. The PI can add other faculties as co-PIs or senior personnel of the proposal. PI and co-PIs may come from different departments and colleges. Before a proposal is submitted to the target funding agency by the research office (not the PI or co-PI) on behalf of the institution, it must go through an approval process. Generally, it needs to be approved by all department chairs of the PI and co-PIs, the college deans, business managers, and IRB office if necessary (e.g., when human

subjects and animals are involved in the project). As the above workflow implies complex authorization issues, GPMS was developed as an exemplar open source application of XACML (eXtensible Access Control Markup Language) [14]. XACML is a standard language for attribute-based access control (ABAC) policies. ABAC defines access control decisions according to fine-grained attributes of subjects, resources, actions, and environments.

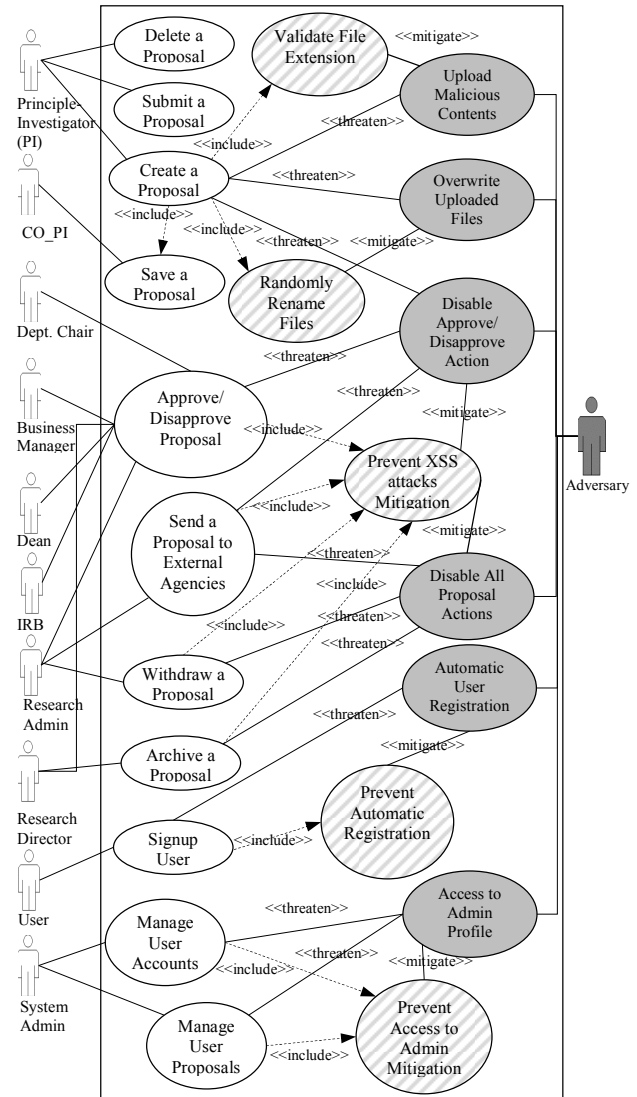


Figure 6. Use case/misuse case/mitigation use case diagram for GPMS

The main technologies and tools for implementing GPMS include Balana (an open source XACML Implementation), RESTful Services, MongoDB (NoSQL database), and Morphia (a lightweight library for mapping Java objects to and from MongoDB database). In addition, GPMS has implemented advanced access control techniques such as obligations and delegations. When the security testing was conducted, GPMS had 24,128 lines of Java code and 84 classes.

Even though security was considered from the beginning, the development of GPMS focused on access control policies. The functions of GPMS are mainly elicited through use cases. Misuse case modeling was not used in the GPMS project until this case study was conducted. The main use cases for various actors (e.g., PI, co-PI, senior personnel, chair, dean, business manager, IRB, research administrator, research director, etc.) include create a proposal, submit a proposal, delete a proposal, save a proposal, approve a proposal, disapproval a proposal, withdraw a proposal, and archive a proposal.

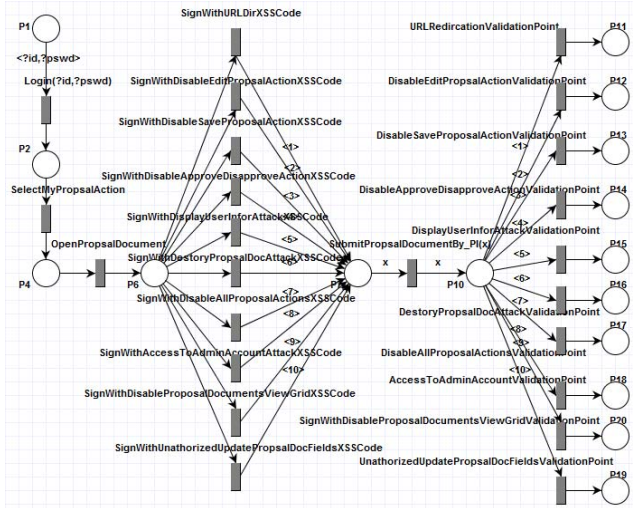


Figure 7. A security test model in the GPMS case study

Based on the 19 use cases of the GPMS project, we identified 30 misuse cases. Figure 6 shows a portion of the use case/misuse case/mitigation use case diagram for GPMS. 14 security test models were created from the use cases, misuse cases, and mitigation use cases. Figure 7 presents the security test model that is corresponding to the “Submit Proposal” use case, and the corresponding misuse and mitigation use cases.

Based on use-case centered approach, 14 security test models were created, which yield 147 security test cases generated by MISTA. These tests have revealed at least 24 vulnerabilities in the GPMS implementation. The vulnerabilities are related to 16 of the original use cases. They can be exploited to disable legitimate proposal actions, update proposal document fields without permission, and deny of service for uploading large files. Based on the security tests, the GPMS team is currently improving the GPMS implementation. This case study has demonstrated that our security testing approach is very effective.

VI. RELATED WORK

Our work is related to software security testing and security requirements analysis. Existing software security testing research falls into three categories: security policy testing, attack-based penetration testing, and fuzzing. Security policy testing focuses on whether the software under test conforms to the specified access control policies. For example, Masood et al. [15] used finite state machines for model-based test

generation of role-based access control (RBAC) policies. Jürjens [16] proposed a method for generating test sequences from UMLsec models to find implementation vulnerabilities. Julliand et al. [17] generated security tests by re-using the functional test model together with a new access control model. Different from security policy testing, penetration testing focuses on whether the software under test is subject to potential attacks. Thus, it is usually based on identification and specification of security threats. Wang et al. [4] proposed to derive security tests from security threats specified by sequence diagrams. Marback et al. [5] proposed to generate security tests from attack trees, which represent threat models of software. As attack trees and sequence diagrams are semi-formal methods, security tests generated from attack trees and sequence diagrams are often incomplete test specifications. To address this issue, MISTA [6][7] was developed to derive complete security tests from formal threat models specified as PrT nets. Even though STRIDE is a useful threat classification for constructing formal threat models, MISTA itself does not prescribe specific methodologies for creating threat models from software artifacts. The contribution of this paper is to provide a methodology for building threat models through misuse case modeling. Different from security policy testing and penetration testing, fuzzing [18] is a method for discovering software vulnerability by providing unexpected inputs and monitoring for exceptions. Traditional fuzzers create test cases by applying random mutations on existing data samples or by generating random invalid inputs from the specification of a protocol or data format. SAGE [19] is an alternative white box fuzz testing approach based on symbolic execution and dynamic test generation. It has found many new security defects in various Windows applications. It is worth investigating how fuzzing can be integrated with security policy testing and attack-based penetration testing.

The misuse case modeling approach used for security testing in this paper originates from Sindre and Opdahl’s work [8], which is an extension to use case modeling for requirements analysis. Alexander applied misuse cases beyond elicitation of security requirements, such as general “-ility” requirements [20]. McDermontt and Fox proposed abuse cases for security requirements analysis [21]. Lin et al [22] used abuse frames to determine the scope of the security problem and analyze the security requirements. Our approach in this paper bridges the gap between security requirements analysis with misuse case modeling and security testing. It can be adapted to other existing methods for security requirements analysis.

VII. CONCLUSIONS

We have presented an approach for building security test models from the artifacts of use case modeling for software requirements analysis, i.e., use cases, misuse cases, and mitigation use cases. These security test models can be used to generate security tests automatically for exercising the software under test. Our case studies have shown that the security tests resulted from our approach are effective in discovering potential vulnerabilities that violate security requirements.

In our current approach, we manually create PrT nets of security test models from the descriptions of use/misuse cases

and mitigation use cases. Our future work will develop a toolkit for editing templates and descriptions of use/misuse cases and mitigation use cases, and converting them into security test models in a semi-automatic way. This will facilitate maintaining traceability among use/misuse cases and mitigation use cases, and between system requirements and security test models. Another direction of future research is to investigate how to build security test models from software requirements elicited by user stories and acceptance criteria in agile software development process. How to deal with security threats and security requirements with user stories and acceptance criteria remains largely open.

ACKNOWLEDGMENT

This work was supported in part by the National Science Foundation under grant CNS 1522847.

REFERENCES

- [1] P. Mueller, S. M. Thampi, Md Z. A. Bhuiyan, R. Ko, R. Doss, J. M. A. Calero, *Security in Computing and Communications*, Springer Singapore, DOI: 10.1007/978-981-10-2738-3.
- [2] D. Xu, "Software security," *Wiley Encyclopedia of Computer Science and Engineering*, B. W. Wah (Editor-In-Chief), Volume 5, pp. 2703-2716, John Wiley & Sons, Inc., Hoboken, NJ, January 2009.
- [3] B. Potter, B. Allen, G. McGraw, "Software security testing," *IEEE Security & Privacy*, (Sept./Oct. 2004), pp. 32-36.
- [4] L. Wang, W.E. Wong, D. Xu, D. "A threat model driven approach for security testing," In *The 3rd International Workshop on Software Engineering for Secure Systems* (Minneapolis, USA, May 2007). SESS'07, in conjunction with ICSE'07.
- [5] A. Marback, H. Do, K. He, S. Kondamari, D. Xu, "A threat model-based approach to security testing," *Software: Practice and Experience*, 43, 2 (Feb. 2013), pp. 241-258.
- [6] D. Xu, M. Tu, M. Sanford, L. Thomas, D. Woodraska, and W. Xu, "Automated security test generation with formal threat models," *IEEE Transactions on Dependable and Secure Computing*, 9, 4 (July/August 2012), pp. 525-539.
- [7] D. Xu, W. Xu, M. Kent, L. Thomas, L. Wang, "An automated test generation technique for software quality assurance," *IEEE Transactions on Reliability*, 64, 1 (March 2015), pp. 247-268.
- [8] Sindre, G. and Opdahl, A.L. 2000. "Eliciting security requirements by misuse cases," In *Proc. of TOOLS Pacific* (2000), pp. 120-131.
- [9] I. Alexander, "Misuse cases: Use cases with hostile intent," *IEEE Software* (Jan/Feb 2003), pp. 58-66.
- [10] K. Bittner and I. Spence, *Use Case Modeling*, Object Technology Series, Addison-Wesley, 2003.
- [11] A. Cockburn, *Writing Effective Use Cases*, Addison-Wesley Professional, October 2000.
- [12] A. Shostack, *Threat Modeling: Designing for Security*. Wiley. February 2014.
- [13] M. Munakami. "Developing an ABAC-Based Grant Proposal Workflow Management System," MS Thesis, Boise State University Department of Computer Science, December 2016.
- [14] OASIS. 2013. eXtensible Access Control Markup Language (XACML) Version 3.0, <http://www.oasisopen.org/committees/xacml/>
- [15] A. Masood, A. Ghafoor, A. Mathur. "Conformance testing of temporal role-based access control systems," *IEEE Trans. on Dependable and Secure Computing*, vol. 7, no. 2, pp. 144-158, 2010.
- [16] J. Jürjens. "Model-based security testing using UMLsec," *Electronic Notes in Theoretical Computer Science (ENTCS)*, 220(1):93-104, December 2008.
- [17] J. Julliand, P.A. Masson, R. Tissot. "Generating security tests in addition to functional tests," *Proc. of the Third International Workshop on Automation of Software Test*, pp. 41-44, 2008.
- [18] M. Sutton. *Fuzzing: Brute Force Vulnerability Discovery*, 1st edition, Addison-Wesley Professional, 2007.
- [19] P. Godefroid, M. Y. Levin, D. Molnar. "SAGE: Whitebox fuzzing for security testing," *Communications of the ACM*, Volume 55, Number 3, pp.40-44, March 2012.
- [20] I. Alexander. "Misuse Cases: Use Cases with Hostile Intent," *IEEE Software*, pp. 58-66, January/February 2003.
- [21] J. McDermott and C. Fox. "Using Abuse Case Models for Security Requirements Analysis," In *Proc. of the 15th Annual Computer Security Application Conference*, pp. 55-66, 1999.
- [22] L. Lin, B. Nuseibeh, D. Ince, M. Jackson, and J. Moffett, "Introducing Abuse Frames for Analysing Security Requirements," In *Proc. of the 11th IEEE International Conference on Requirements Engineering (RE03)*, pp. 371-372, 2003.